

AGENTIC AI SPECIALIST COURSE CURRICULAM

LEVEL 2 — ADVANCED AGENTIC AI CERTIFICATION (6 WEEKS) ENHANCED 6-WEEK ADVANCED CURRICULUM

AGENTIC AI SPECIALIST COURSE - LEVEL 2 ADVANCED AGENTIC AI CERTIFICATION (6 WEEKS)

COURSE OVERVIEW

Program Prerequisites

Minimum Requirements

- Python fundamentals (functions, classes, error handling)
- Basic API knowledge (REST, HTTP methods)
- Basic web development familiarity (HTML, CSS, JavaScript basics)
- Familiarity with JSON, HTTP requests
- Understanding of asynchronous programming concepts
- Git and version control basics

Recommended (But Optional)

- Prior ML or AI exposure
- Experience using OpenAI, Anthropic or similar LLM APIs
- Basic understanding of vector databases
- Docker and containerization concepts
- Cloud platform experience (AWS/GCP/Azure)

Course Outcomes

After completing the Advanced course, learners will be able to:

Technical Competencies

1. Design and implement complex multi-agent systems with fault tolerance
2. Build production-grade RAG systems with hybrid retrieval and validation
3. Implement advanced agent coordination patterns (auctions, consensus, swarms)
4. Deploy scalable distributed agent systems with proper observability
5. Fine-tune and distill LLMs for specialized agent tasks
6. Implement comprehensive evaluation and validation frameworks
7. Build secure sandboxed code execution environments
8. Optimize agent systems for cost, latency, and reliability
9. Create multi-modal agents with vision and web automation capabilities
10. Apply advanced reasoning techniques (Tree-of-Thoughts, Graph-of-Thoughts)

Professional Outcomes

- Transition to Senior/Lead Agentic AI Developer roles
- Architect enterprise-grade AI systems
- Contribute to cutting-edge agent research
- Build production-ready AI products
- Lead AI engineering teams

CURRICULUM STRUCTURE

Week 7 — Multi-Agent Systems Architecture & Emergent Behaviors

Focus: Complex agent coordination, consensus mechanisms, and distributed intelligence

Week 8 — Code Execution, Automation & Multi-Modal Agents

Focus: Secure sandboxing, web automation, vision-language integration

Week 9 — Advanced RAG, Validation & Knowledge Engineering

Focus: Hybrid retrieval, knowledge graphs, self-reflective systems

Week 10 — Fine-Tuning, Distillation & Tool Engineering

Focus: Model optimization, synthetic data, production tools

Week 11 — Distributed Systems, Observability & Production ML

Focus: Scalable infrastructure, monitoring, cost optimization

Week 12 — Advanced Capstone Week

Focus: End-to-end production system with novel contributions

WEEK 7 — MULTI-AGENT SYSTEMS ARCHITECTURE & EMERGENT BEHAVIORS

Day 1: Multi-Agent Architecture Fundamentals

Topics Covered:

- Hierarchical vs Flat Agent Topologies
- Agent Communication Protocols (ACL, FIPA standards)
- Synchronous vs Asynchronous Agent Conversations
- Agent Mesh Networks with Dynamic Role Assignment
- Byzantine Fault Tolerance in Agent Systems
- Planner/Solver/Reviewer/Orchestrator Roles
- Agent State Management and Persistence
- Distributed Agent Coordination Patterns
- Agent Discovery and Service Registration
- Dynamic Agent Lifecycle Management

Exercise: Design a 5-agent hierarchy with Byzantine fault tolerance where agents can dynamically reassign tasks based on workload using a custom priority queue system. Implement consensus mechanisms for critical decisions. The system should handle 2 faulty agents without degradation.

Day 2: AutoGen Advanced Patterns & Negotiation

Topics Covered:

- Conversational Agents with Memory Systems
- Agent-to-Agent Negotiation with Nash Equilibrium
- Task Delegation with Constraint Satisfaction
- Auction-Based Task Allocation Algorithms
- Agent Reputation Scoring Systems
- Game-Theoretic Approaches to Multi-Agent Decision Making
- Coalition Formation Algorithms
- Mechanism Design for Agent Coordination
- Preference Elicitation Techniques
- Strategic Reasoning in Multi-Agent Environments

Exercise: Create a four-agent auction system where agents bid on subtasks based on capability scores. Implement:

- First-price sealed-bid auction mechanism
- Vickrey auction for truthful bidding
- Reputation update mechanism based on task completion quality
- Dynamic reserve price adjustment
- Strategy-proof bidding protocols

Day 3: LangGraph State Machines & Probabilistic Transitions

Topics Covered:

- Directed Acyclic Graphs (DAGs) for Deterministic Workflows
- Probabilistic State Transitions with Confidence Scoring
- Cycles, Loops, and Conditional Branching
- State Persistence and Checkpointing Mechanisms
- Graph Optimization using Dynamic Programming
- Adaptive Graph Structure Modification
- Markov Decision Processes in Agent Workflows
- Partially Observable State Transitions
- Graph Neural Networks for Workflow Optimization
- Temporal Dependencies in State Machines

Exercise: Build a stateful agent workflow that:

- Adapts execution paths based on intermediate results
- Implements 3+ conditional branches and 2+ recovery loops
- Uses probabilistic transitions based on confidence scores (threshold: 0.7)
- Maintains checkpoints for rollback capabilities
- Dynamically modifies graph structure based on performance metrics
- Handles state recovery after failures

Day 4: CrewAI & Specialized Agent Teams

Topics Covered:

- Role Specialization with Skill Matrices
- Task Orchestration with Dependency Graphs
- Agent Skill Embeddings for Automatic Role Assignment
- Leader, Researcher, Analyst, Writer, Validator Roles
- Agent Fatigue Models and Load Balancing
- Skill Transfer Learning Between Agents
- Team Formation Optimization
- Cross-Functional Team Coordination
- Agent Capability Discovery
- Dynamic Team Composition

Exercise: Build a 6-agent research system:

- **Research Coordinator:** orchestrates workflow with task prioritization
- **Data Gatherer:** collects information from multiple sources
- **Domain Expert:** validates technical accuracy with confidence scores
- **Analyst:** performs quantitative analysis and generates insights
- **Writer:** synthesizes findings into coherent reports
- **Fact-Checker:** validates claims with citations and source verification

Each agent should request clarifications from others and maintain conversation history with context windows.

Day 5: Multi-Agent Coordination & Consensus Mechanisms

Topics Covered:

- Byzantine Fault Tolerance in Agent Systems
- Conflict Resolution with Voting Algorithms (Majority, Weighted, Borda Count)
- Raft Consensus for Distributed Agents
- PBFT (Practical Byzantine Fault Tolerance) Implementation
- Performance Monitoring and Agent Observability
- Emergent Behavior Detection and Control
- Swarm Intelligence Principles

- Distributed Consensus Algorithms (Paxos, Raft)
- Conflict-Free Replicated Data Types (CRDTs)
- Vector Clocks for Causality Tracking
- Gossip Protocols for Information Dissemination

Exercise: Implement a 7-agent voting system with:

- Byzantine fault tolerance (tolerates 2 faulty agents)
- Weighted voting based on agent confidence scores and historical accuracy
- Raft consensus for state synchronization
- Automatic detection of byzantine behavior patterns
- Recovery mechanisms for faulty agents
- Performance metrics: consensus latency, fault detection accuracy

Weekend Assignment: Advanced Multi-Agent Debate System

Build a comprehensive Multi-Agent Debate & Decision System with:

Agents:

1. **Proposition Agent:** defines thesis with supporting framework
2. **Research Agent:** gathers evidence from multiple sources
3. **Advocate Agent:** constructs arguments for the proposition
4. **Skeptic Agent:** identifies weaknesses and counterarguments
5. **Expert Panel:** 3 domain-specific agents with specialized knowledge
6. **Judge Agent:** evaluates arguments with weighted scoring
7. **Meta-Analyst:** critiques debate quality and methodology

Deliverables:

- Full conversation trace with reasoning chains and confidence scores
- Argument graph visualization with support/attack relationships
- Tool usage logs with API calls and data sources
- Final verdict with justification and confidence intervals (95% CI)
- Argument mining and claim extraction module
- Counter-argument generation with evidence scoring
- Bias detection in argumentation (political, confirmation, anchoring)
- Performance report: consensus time, argument quality scores

WEEK 8 — CODE EXECUTION, AUTOMATION & MULTI-MODAL AGENTS

Day 1: Secure Sandboxed Code Execution

Topics Covered:

- Docker-Based Python Sandboxing with Resource Limits
- CPU, Memory, and Time Constraints
- WebAssembly for Client-Side Code Execution

- Import Whitelisting and AST (Abstract Syntax Tree) Validation
- Static Code Analysis Before Execution (Bandit, Pylint)
- Runtime Security Monitoring
- Containerization Best Practices for Code Execution
- Security Policies and SELinux/AppArmor
- Code Signing and Verification
- Secure Inter-Process Communication

Exercise: Build a code execution sandbox that:

- Detects and blocks infinite loops using AST analysis (max depth: 100)
- Prevents excessive memory usage with resource monitors (limit: 512MB)
- Blocks forbidden imports (os, subprocess, socket) and system calls
- Implements graduated permissions based on code complexity
- Logs all execution attempts with security scoring (0-100 scale)
- Handles timeout scenarios gracefully (max execution: 30s)
- Provides detailed error messages without exposing system internals

Day 2: Code-Writing & Self-Debugging Agents

Topics Covered:

- Tool Calling for Code Generation
- Syntax Tree Manipulation for Code Repair
- Test-Driven Development with Agents
- Program Synthesis from Natural Language Specifications
- Automated Test Case Generation
- Output Validation with Unit Tests
- Symbolic Execution for Bug Detection
- Fuzzing for Security Vulnerabilities
- Semantic Code Search
- Code Clone Detection and Refactoring

Exercise: Create a self-debugging agent that:

- Writes Python code from natural language specifications
- Generates comprehensive unit tests with edge cases (min 10 tests per function)
- Runs tests and analyzes failures with stack traces
- Uses syntax tree manipulation to fix bugs automatically
- Iteratively improves code quality (max 5 iterations)
- Provides code coverage reports (target: >80% coverage)
- Generates complexity metrics (cyclomatic complexity, maintainability index)
- Suggests performance optimizations with profiling data

Day 3: Advanced Web Automation & Anti-Detection

Topics Covered:

- Playwright with Anti-Detection Techniques
- Handling CAPTCHA and Bot Protection (2Captcha, Anti-Captcha)
- Dynamic Selector Strategies (XPath, CSS, AI-Based)
- Headless Browser Fingerprinting Evasion
- State Management Across Navigation
- Proxy Rotation and Session Management
- Browser Automation Patterns
- DOM Mutation Observers
- Shadow DOM Navigation
- Browser Extension Integration

Exercise: Build a resilient web scraper that:

- Adapts selectors when page structure changes using visual similarity (threshold: 0.85)
- Handles JavaScript-rendered content with dynamic waiting strategies
- Implements CAPTCHA solving with fallback mechanisms (3 services)
- Uses proxy rotation to avoid IP blocking (10+ proxy pool)
- Stores session cookies and handles authentication flows
- Validates extracted data against predefined JSON schemas
- Implements retry logic with exponential backoff (max 5 retries)
- Handles rate limiting with adaptive delays

Day 4: Intelligent Browsing Agents with Planning

Topics Covered:

- Multi-Page Navigation Planning
- Markov Decision Processes for Browsing Strategy
- Form-Filling with Context Awareness
- Browser Agent That Learns from Failed Attempts
- Structured Data Extraction with Schema Validation
- Visual Page Understanding with Computer Vision
- Reinforcement Learning for Web Navigation
- Page Embedding and Similarity Search
- Automated Workflow Discovery
- Browser Performance Optimization

Exercise: Create a job aggregator that:

- Learns extraction patterns from 5 different career sites
- Automatically adapts to layout changes using visual diff (OpenCV)

- Validates extracted data against JSON schema (required fields: title, company, location, salary)
- Stores failed attempts and learns recovery strategies
- Implements incremental crawling with change detection (hash-based)
- Generates structured output with confidence scores (per field)
- Handles pagination automatically (max 100 pages per site)
- Deduplicates job postings using fuzzy matching (threshold: 0.9)

Day 5: Multi-Modal Vision-Language Agents

Topics Covered:

- Image Captioning with Object Detection (YOLO, DETR)
- OCR + Layout Understanding (Tesseract, EasyOCR)
- Visual Question Answering (VQA) with ViLT, BLIP
- Screenshot-to-Code Generation (pix2code, Sketch2Code)
- Video Frame Analysis for Temporal Reasoning
- Audio-Visual Fusion for Multi-Modal Understanding
- Cross-Modal Retrieval
- Multi-Modal Embeddings
- Vision Transformers (ViT)
- Attention Mechanisms in Multi-Modal Models

Exercise: Build a comprehensive UI testing agent that:

- Takes screenshots of web applications at multiple resolutions
- Identifies all interactive elements (buttons, inputs, links, dropdowns)
- Generates Selenium/Playwright test scripts automatically
- Predicts expected outcomes for interactions
- Detects UI anomalies and accessibility issues (WCAG 2.1 compliance)
- Generates visual regression tests with pixel-perfect comparison
- Creates documentation with annotated screenshots
- Supports responsive design testing (mobile, tablet, desktop)

Weekend Assignment: Autonomous Research & Reporting Bot

Build a comprehensive Multi-Modal Research System that:

Core Features:

- Accepts complex research queries in natural language
- Plans multi-step search strategy with source diversity (min 5 different domains)
- Executes web searches with adaptive query refinement
- Takes screenshots of key sources for visual evidence
- Extracts and validates data with cross-referencing (min 3 sources per claim)

- Performs OCR on images and PDFs with accuracy >95%
- Generates visualizations (charts, graphs, network diagrams) using matplotlib/plotly
- Produces final report with citations and confidence scores (per section)
- Creates executive summary with key findings (max 500 words)

Advanced Requirements:

- Implement intelligent retry logic that learns from failures (max 3 retries per query)
- Use multi-modal reasoning (text + images + tables)
- Detect and handle paywalls with alternative source finding
- Generate argument maps visualizing claim relationships (NetworkX)
- Implement source credibility scoring (domain authority, recency, citations)
- Handle temporal queries (compare information over time with timeline visualization)
- Support multiple output formats (PDF, Markdown, HTML)

WEEK 9 — ADVANCED RAG, VALIDATION & KNOWLEDGE ENGINEERING

Day 1: Hybrid Retrieval Systems

Topics Covered:

- Dense (Semantic) + Sparse (BM25) Fusion
- Learned Sparse Retrieval (SPLADE, uniCOIL)
- Cross-Encoder Re-Ranking
- Reciprocal Rank Fusion (RRF) Algorithms
- Query Expansion with Pseudo-Relevance Feedback
- Query Reformulation with LLMs
- Multi-Vector Retrieval Strategies
- ColBERT for Late Interaction
- Dense Passage Retrieval (DPR)
- Approximate Nearest Neighbors (ANN) with HNSW
- Embedding Model Selection and Fine-Tuning

Exercise: Implement a hybrid retrieval system that:

- Automatically adjusts dense/sparse weights based on query type (factual: 0.7/0.3, conceptual: 0.5/0.5)
- Uses query classification (factual vs conceptual vs procedural) with 90%+ accuracy
- Implements cross-encoder re-ranking with score calibration (top-20 → top-5)

- Applies query expansion with LLM-generated variations (3-5 variations per query)
- Handles multi-hop queries with iterative retrieval (max 3 hops)
- Measures retrieval quality with NDCG@10, MRR, MAP
- Implements caching for repeated queries (Redis)

Day 2: Multi-Hop & Chain-of-Thought Retrieval

Topics Covered:

- Iterative Retrieval with Reasoning Traces
- Beam Search for Retrieval Chains
- Long-Context Stitching with Summarization
- Retrieval-Augmented Thought (RAT) Patterns
- Question Decomposition Strategies
- Self-Ask Prompting for Complex Queries
- Iterative Refinement with Feedback Loops
- Graph-Based Retrieval Paths
- Attention-Guided Retrieval
- Contextual Chunk Selection
- Dynamic Context Window Management

Exercise: Build a multi-hop agent that answers:

Query: "Compare the economic policies of Company A and Company B over 3 years, and explain which factors led to Company A's superior growth."

Requirements:

- Decompose query into sub-questions (min 5 sub-questions)
- Implement beam search for optimal retrieval path (beam width: 3)
- Extract relevant sections from financial reports (10-K filings)
- Synthesize information across multiple documents (min 6 documents)
- Generate comparison with supporting evidence and confidence scores
- Validate claims with source citations (hyperlinked)
- Provide confidence scores for each claim (0-100 scale)
- Handle conflicting information with explicit disambiguation

Day 3: Knowledge Graphs & Entity-Centric RAG

Topics Covered:

- Neo4j/GraphDB Fundamentals
- Entity Linking and Disambiguation (BLINK, ELQ)
- Temporal Knowledge Graphs with Time-Aware Reasoning
- Relationship Extraction with LLMs

- Knowledge Graph Embeddings (TransE, RotatE, ComplEx)
- Multi-Hop Reasoning over Knowledge Graphs
- Knowledge Graph Completion with Neural Methods
- Ontology Design and Schema Mapping
- Knowledge Graph Alignment
- Rule Mining for Knowledge Graphs
- Uncertainty Quantification in KGs

Exercise: Build a dynamic knowledge graph that:

- Extracts entities and relationships from 30 news articles
- Implements temporal reasoning (events over time with timestamps)
- Handles entity disambiguation with confidence scores (threshold: 0.8)
- Supports complex graph traversal queries (Cypher queries)
- Uses graph embeddings for link prediction (accuracy >75%)
- Answers multi-hop queries with reasoning paths (max 4 hops)
- Visualizes subgraphs for explanations (using D3.js or PyVis)
- Tracks entity evolution over time

Day 4: Graph-RAG & Reasoning Over Structures

Topics Covered:

- Hybrid Vector + Graph Retrieval
- Cypher Query Generation from Natural Language
- Graph Neural Networks for Reasoning
- Subgraph Extraction for Context
- Multi-Hop Path Ranking
- Community Detection for Topic Clustering
- Graph Attention Networks for Node Importance
- Graph Convolutional Networks (GCN)
- Heterogeneous Graph Neural Networks
- Knowledge Graph Reasoning
- Graph-to-Text Generation

Exercise: Create a scientific paper KG with:

- 100 papers, authors, institutions, and citations
- Relationship types: cites, authored-by, affiliated-with, builds-on
- Temporal information: publication dates, citation evolution

Agent capabilities:

- "Find papers connecting RL and drug discovery with citation chain"
- "Identify emerging research trends in the last 2 years" (using citation velocity)

- "Discover potential collaborations between institutions" (using collaboration graphs)
- "Generate research summaries with key contributions"
- "Detect citation patterns and influential papers" (PageRank, betweenness centrality)
- Support natural language queries with Cypher translation

Day 5: Self-Reflective & Validated RAG

Topics Covered:

- Query Rewriting with Chain-of-Thought
- Hallucination Detection with NLI Models (RoBERTa, DeBERTa)
- Multi-Pass Retrieval with Refinement
- Uncertainty Quantification in Retrieved Context
- Self-Consistency Checks with Multiple Samples
- Answer Validation with Fact-Checking
- Retrieval Quality Metrics (NDCG, MRR, MAP)
- Calibration of Confidence Scores
- Active Learning for Query Improvement
- Adversarial Question Generation
- Robustness Testing for RAG Systems

Exercise: Implement a self-correcting RAG system that:

- Generates initial answer from retrieved context
- Runs fact-checking against sources with NLI (entailment score >0.8)
- Identifies inconsistencies and hallucinations (per sentence)
- Retrieves additional context for uncertain claims (confidence <0.7)
- Regenerates answer with corrections (max 3 iterations)
- Provides confidence scores per sentence (0-100 scale)
- Highlights supporting evidence for each claim with source links
- Implements self-consistency with 3 answer samples (majority vote)
- Measures calibration error (ECE <0.1)

Day 6: ML & LLM Validation Techniques (BONUS)

Cross-Validation for Agentic Systems:

- K-Fold Cross-Validation for Agent Datasets
- Stratified Sampling for Imbalanced Agent Tasks
- Time-Series Cross-Validation for Temporal Agent Data
- Leave-One-Group-Out for Multi-Domain Agents
- Nested Cross-Validation for Hyperparameter Tuning

- Bootstrap Aggregating (Bagging)
- Monte Carlo Cross-Validation
- Cross-Validation for Dependent Data
- Evaluation Metrics Selection

LLM-Specific Validation Methods:

- Monte Carlo Dropout for Uncertainty Estimation
- Semantic Similarity Validation (BERTScore, MoverScore, BARTScore)
- Self-Consistency Checks (N Answers, Agreement Measurement)
- Contrastive Validation with Adversarial Examples
- Human-in-the-Loop Validation Frameworks
- Perplexity-Based Quality Assessment
- Bayesian Deep Learning for Uncertainty
- Ensemble Methods for LLMs
- Active Learning Strategies
- Calibration Techniques

Novel Validation Approaches:

- Retrieval Quality Metrics (NDCG, MRR, MAP, Precision@k)
- Hallucination Detection Using Entailment Models
- Agent Trajectory Validation (Logical Step Verification)
- Multi-Reference Evaluation (BLEU, ROUGE, METEOR for Generation)
- Causal Validation (Reasoning Chain Verification)
- Factuality Scores with External Knowledge Bases
- Adversarial Robustness Testing
- Explainable AI for Validation
- Meta-Learning for Validation
- Transfer Learning for Evaluation
- Multi-Task Validation

Exercise: Build a comprehensive validation pipeline that:

Dataset Preparation:

- Creates 500-example agent task dataset
- Implements stratified splits for balanced evaluation (80/10/10)
- Handles time-series data with temporal splits (no data leakage)

Cross-Validation:

- Runs 10-fold cross-validation with multiple metrics
- Implements nested CV for hyperparameter search
- Computes confidence intervals for metrics (95% CI)

Self-Consistency:

- Generates 5 answers per question

- Measures agreement with semantic similarity (BERTScore)
- Selects best answer via majority vote or consensus

Hallucination Detection:

- Uses NLI models for fact-checking (RoBERTa-large-MNLI)
- Compares against source documents
- Generates hallucination scores per sentence (0-1 scale)

Semantic Accuracy:

- Computes BERTScore for generated text
- Evaluates multi-reference outputs
- Measures semantic drift over conversations

Reasoning Validation:

- Builds causal graphs for reasoning chains
- Validates logical consistency
- Detects circular reasoning

Comprehensive Metrics:

- Retrieval: NDCG@10, MRR, MAP
- Generation: BLEU, ROUGE, METEOR, BERTScore
- Factuality: Entailment scores, citation accuracy
- Uncertainty: Calibration curves, ECE (Expected Calibration Error <0.1)

Weekend Assignment: Validated Hybrid RAG System

Build a production-ready Hybrid RAG System with:

System Components:

Retrieval Layer:

- Dense retrieval (sentence-transformers: all-MiniLM-L6-v2)
- Sparse retrieval (BM25 with custom tokenization)
- Graph-based retrieval (Neo4j with entity linking)
- Hybrid fusion with learned weights (optimized via grid search)

Query Understanding:

- Query classification (factual/conceptual/procedural) with 90%+ accuracy
- Query reformulation with LLM (GPT-4 or Claude)
- Sub-query decomposition for complex questions
- Intent detection with confidence scoring

Validation Layer:

- Self-consistency: generate 3 answers, pick majority
- Hallucination detection with NLI models (DeBERTa-large-MNLI)
- Confidence scoring with uncertainty quantification (Monte Carlo Dropout)
- Claim verification against sources (entailment >0.8)

Quality Metrics:

- Retrieval quality: NDCG@10 >0.7, MRR >0.6, MAP >0.5
- Answer quality: BERTScore >0.85, ROUGE-L >0.4
- Factuality: Entailment scores >0.8
- Calibration: ECE <0.1, reliability diagrams

Evaluation:

- Test on 100 questions from diverse domains (10 domains, 10 questions each)
- Report accuracy, hallucination rate (<5%), confidence calibration
- Provide ablation study (dense vs sparse vs hybrid)
- Generate error analysis with failure categories
- Compare against baseline RAG system (performance improvement >20%)

Deliverables:

- Working system with API endpoint (FastAPI)
- Evaluation report with metrics and visualizations
- Ablation study results
- Error analysis with examples
- Documentation with deployment guide

WEEK 10 — FINE-TUNING, DISTILLATION & TOOL ENGINEERING**Day 1: Advanced Fine-Tuning Techniques****Topics Covered:**

- LoRA, QLoRA, DoRA (Weight-Decomposed Low-Rank Adaptation)
- Mixture of LoRAs (MoLoRA) for Multi-Task Learning
- Prompt Tuning vs Prefix Tuning vs Full Fine-Tuning
- RLHF (Reinforcement Learning from Human Feedback)
- DPO (Direct Preference Optimization)
- Dataset Formats and Preprocessing (JSONL, Parquet)
- Instruction Tuning with Synthetic Data
- Parameter-Efficient Fine-Tuning (PEFT)
- Adapter Layers and Bottleneck Adapters
- Multi-Task Learning Strategies
- Catastrophic Forgetting Prevention

Exercise: Fine-tune a 7B model (Llama-2-7B or Mistral-7B) using:

- QLoRA for memory efficiency (4-bit quantization)
- Custom domain-specific dataset (1000+ examples, e.g., medical Q&A)
- Compare with prompt tuning baseline (measure performance gap)
- Measure perplexity (target: <10), task accuracy (target: >85%), and inference speed

- Implement early stopping with validation metrics (patience: 3 epochs)
- Analyze which layers benefit most from fine-tuning (visualization)
- Evaluate on out-of-distribution test set (measure generalization)
- Track GPU memory usage and training time

Day 2: Data Engineering for LLMs

Topics Covered:

- Synthetic Data Generation with GPT-4
- Active Learning for Efficient Labeling
- Data Augmentation Strategies (Back-Translation, Paraphrasing)
- Deduplication and Quality Filtering
- Data Mixture Optimization
- Instruction Tuning Formats (Alpaca, ShareGPT)
- Curriculum Learning for Data Ordering
- Data Contamination Detection
- Privacy-Preserving Data Augmentation
- Bias Detection in Training Data
- Multilingual Data Balancing

Exercise: Generate comprehensive synthetic dataset:

Task: Create 1000 examples for "technical documentation Q&A"

Requirements:

- Use GPT-4 for diverse question-answer pairs
- Implement quality filters:
 - Answer length constraints (50-500 words)
 - Factual consistency checking (NLI score >0.85)
 - Diversity scoring with embeddings (cosine similarity <0.8 between examples)
 - Toxicity detection (Perspective API score <0.5)
- Apply deduplication with MinHash LSH (Jaccard similarity threshold: 0.9)
- Create train/val/test splits with stratification (70/15/15)
- Augment with back-translation (English → French → English, 20% of dataset)
- Generate difficulty scores for curriculum learning (easy → hard ordering)
- Document data statistics and quality metrics

Day 3: Model Distillation & Compression

Topics Covered:

- Knowledge Distillation from Large to Small Models
- Quantization Techniques (INT8, INT4, GPTQ, AWQ)

- Pruning and Structured Sparsity
- Distillation with Chain-of-Thought Reasoning
- Layer-wise Distillation Strategies
- Model Merging (SLERP, TIES-Merging)
- Neural Architecture Search (NAS)
- AutoML for Model Compression
- Hardware-Aware Optimization
- Inference Optimization Techniques

Exercise: Distill GPT-4's reasoning into a 3B model:

- Generate 2000 chain-of-thought examples from GPT-4 (covering 10+ domains)
- Train student model with knowledge distillation loss (temperature: 2.0)
- Implement layer-wise distillation (match intermediate representations)
- Apply quantization (INT8) for deployment (using llama.cpp or GPTQ)
- Evaluate reasoning quality on benchmark tasks (GSM8K, MMLU)
- Measure latency (target: <100ms per token) and memory usage (target: <6GB)
- Compare against baseline 3B model (performance gap <10%)
- Generate distillation efficiency report (speedup, memory reduction)

Day 4: Building Production-Grade Tools

Topics Covered:

- Tool Schema Design and Validation (OpenAPI, JSON Schema)
- Tool Versioning and Backward Compatibility
- Error Handling and Fallback Strategies
- Tool Usage Analytics and Optimization
- Tool Chaining and Composition
- Dynamic Tool Discovery and Registration
- Tool Security and Access Control
- Tool Monitoring and Logging
- Tool Rate Limiting
- Tool Caching Strategies
- Tool Performance Profiling

Exercise: Create a production tool registry system:

- Define 10 tools with OpenAPI specs (e.g., calculator, web_search, database_query)
- Implement tool versioning (v1, v2 compatibility with deprecation warnings)
- Build tool discovery API with semantic search (embedding-based)
- Create tool usage analytics dashboard (top tools, error rates, latency)
- Implement rate limiting per tool (e.g., 100 requests/min)

- Add circuit breakers for failing tools (failure threshold: 50% in 1 min)
- Generate tool usage recommendations based on context
- Support tool composition (chaining with dependency resolution)
- Implement authentication and authorization (API keys, role-based access)

Day 5: Meta-Learning & Reasoning Amplification

Topics Covered:

- Self-Consistency Sampling with Majority Vote
- Chain-of-Thought Distillation
- Multi-Agent Debate for Improved Reasoning
- Tree-of-Thoughts Exploration (ToT)
- Graph-of-Thoughts Reasoning (GoT)
- Algorithm Distillation for In-Context Learning
- Self-Refine Iterative Improvement
- Meta-Learning Algorithms (MAML, Reptile)
- Few-Shot Learning Strategies
- Reasoning Path Optimization
- Symbolic-Neural Integration

Exercise: Implement Tree-of-Thoughts solver:

- Generate 3 reasoning branches for math problems (GSM8K dataset)
- Evaluate each branch with scoring function (correctness + clarity)
- Implement breadth-first and depth-first search strategies
- Prune low-scoring branches (threshold: score < 0.6)
- Select best solution path (highest cumulative score)
- Compare with standard chain-of-thought (accuracy improvement)
- Measure accuracy (target: >85%) and computational cost (token usage)
- Implement self-consistency as ensemble method (5 samples, majority vote)
- Analyze failure cases and reasoning patterns

Weekend Assignment: Fine-Tune & Validate Specialized Agent

Choose a domain and build end-to-end:

1. Medical Q&A
2. Legal Document Analysis
3. Code Review System
4. Financial Analysis Agent

Deliverables:

1. Model Artifacts:

- Fine-tuned model with LoRA adapters (rank: 16, alpha: 32)

- Model card with architecture details (base model, dataset, hyperparameters)
- Training logs and loss curves (TensorBoard logs)
- Hyperparameter configuration (learning rate, batch size, epochs)

2. Dataset:

- Synthetic + curated (min 2000 examples, 70/15/15 split)

everyday topics are very more and we cannot cover them in 3-4 hours, so give the mostneeded topics alone for each day , and exercise ofr them accordingly)

- Data quality report with statistics (avg length, diversity score, toxicity)
- Deduplication and filtering logs (examples removed, reasons)
- Train/val/test splits with distribution analysis (domain coverage)

3. Evaluation Suite:

- 10-fold cross-validation results (mean accuracy, std dev)
- Hallucination detection scores per example (NLI-based)
- Human evaluation on 100 samples (3 annotators, Fleiss' kappa >0.7)
- Comparison with baseline models (GPT-3.5, base model)
- Confusion matrices and error analysis (false positives, false negatives)

4. Documentation:

- Model card with limitations and biases (documented failure modes)
- A/B test results vs base model (statistical significance, p-value <0.05)
- Inference optimization guide (quantization, batching)
- Deployment recommendations (hardware, scaling strategies)

5. Advanced Analysis:

- Attention visualization for sample inputs (using BertViz or similar)
- Saliency maps for interpretability
- Uncertainty quantification (confidence calibration)
- Robustness testing with adversarial examples (attack success rate <10%)

WEEK 11 — DISTRIBUTED SYSTEMS, OBSERVABILITY & PRODUCTION ML

Day 1: Distributed Agent Architecture

Topics Covered:

- Message Queues (RabbitMQ, Kafka, NATS)
- Event Sourcing for Agent State with CQRS
- Worker Pools and Task Scheduling (Celery, Temporal)
- Saga Pattern for Distributed Transactions
- Circuit Breakers and Bulkheads for Resilience
- Service Mesh for Agent Communication (Istio, Linkerd)

- Distributed Tracing
- Service Discovery
- API Gateway Patterns
- Backpressure Handling

Exercise: Build distributed pipeline with:

- 3 worker types (data processing, LLM inference, validation)
- Kafka for event streaming (3 topics: raw_data, processed_data, validated_data)
- Celery for task queuing (with Redis backend)
- Dead letter queues for failed tasks (retry after 5 min, max 3 retries)
- Retry logic with exponential backoff (initial delay: 1s, max delay: 60s)
- Health checks and auto-scaling (scale up at 80% CPU, scale down at 20% CPU)
- Graceful shutdown handling (drain connections, finish in-flight tasks)
- Implement idempotency for task processing

Day 2: Agent Observability & Monitoring

Topics Covered:

- OpenTelemetry for Distributed Agent Tracing
- Prompt Logging and Analysis with LangSmith
- LangFuse Integration for Production Monitoring
- Cost Tracking per Agent/Tool
- Agent Performance Profiling with Metrics
- Prompt Drift Detection
- Real-Time Alerting for Agent Failures
- Custom Metrics and Dashboards
- Log Aggregation and Analysis
- Anomaly Detection in Agent Behavior
- SLA Monitoring

Exercise: Implement comprehensive observability:

- OpenTelemetry tracing for 5-agent system (spans, traces, context propagation)
- Custom metrics: latency (p50, p95, p99), cost per request, success rate
- Distributed trace visualization (Jaeger UI)
- Log aggregation with ELK stack (Elasticsearch, Logstash, Kibana)
- Identify bottlenecks in execution flow (using trace analysis)
- Set up alerts for latency >2s or errors >5% (PagerDuty/Slack integration)
- Create Grafana dashboard with key metrics (real-time updates)
- Implement cost attribution per user/agent (track token usage)

Day 3: Containerization & Orchestration

Topics Covered:

- Multi-Stage Docker Builds for Optimization
- Container Security Scanning (Trivy, Snyk)
- Kubernetes Deployments, Services, Ingress
- StatefulSets for Agent Memory Persistence
- ConfigMaps and Secrets Management
- Horizontal Pod Autoscaler (HPA) Configuration
- Persistent Volume Claims (PVC) for Agent State
- Helm Charts for Package Management
- GitOps with ArgoCD
- Service Mesh Integration
- Blue-Green and Canary Deployments

Exercise: Deploy 3-agent system on Kubernetes:

- Create multi-stage Dockerfiles (builder stage: compile deps, runtime stage: minimal image)
- Configure StatefulSets for agents with memory (3 replicas)
- Set up PVCs for persistent agent state (ReadWriteMany access mode)
- Implement HPA based on CPU (target: 70%) and custom metrics (queue depth)
- Configure ingress with TLS termination (cert-manager for auto-renewal)
- Set up ConfigMaps for environment-specific settings (dev, staging, prod)
- Implement readiness and liveness probes (HTTP endpoints)
- Use Secrets for sensitive data (API keys, DB passwords)

Day 4: Advanced Scaling Strategies

Topics Covered:

- Model Serving with vLLM / TGI (Text Generation Inference)
- Batched Inference for Efficiency (Dynamic Batching)
- Token Streaming for Low Latency
- Speculative Decoding for Speedup
- KV Cache Optimization and Sharing
- PagedAttention for Memory Efficiency
- Multi-GPU Inference with Tensor Parallelism
- Model Quantization for Inference
- Flash Attention and Other Optimizations
- Request Scheduling Algorithms
- Load Balancing Strategies

Exercise: Deploy high-performance inference system:

- Set up vLLM server with continuous batching (batch size: dynamic, max 32)
- Configure PagedAttention for memory efficiency (block size: 16)
- Implement token streaming for real-time responses (WebSocket or SSE)
- Enable speculative decoding with draft model (Llama-2-70M as draft)
- Benchmark throughput vs latency trade-offs (generate curves)
- Implement request queuing with priorities (priority queue with 3 levels)
- Measure cost per token and optimize (target: <\$0.001 per 1K tokens)
- Test with load: 100 concurrent users, measure p95 latency

Day 5: Cost Optimization & Efficiency

Topics Covered:

- Prompt Compression Techniques (LLMLingua, Gist Tokens)
- Semantic Caching with Embedding Similarity
- Response Caching with TTL and Invalidation
- Model Routing (Small/Large based on Complexity)
- Token Budget Allocation per Agent
- Prompt Token Reduction Strategies
- Inference Cost Tracking and Attribution
- LoRA Adapter Swapping
- Multi-Tenant Model Serving
- Request Deduplication
- Predictive Pre-warming

Exercise: Build intelligent caching and routing:

- Implement semantic cache with FAISS (embedding: all-MiniLM-L6-v2)
- Cache hit/miss metrics tracking (target: >60% hit rate)
- TTL-based cache invalidation (TTL: 1 hour, max cache size: 10K entries)
- Model routing: simple queries → small model (7B), complex → large model (70B)
- Complexity classifier: heuristic (query length, entity count) + learned (fine-tuned BERT)
- Token budget per user session (max 10K tokens per session)
- Cost tracking per request (store in database with user attribution)
- Generate cost optimization report (savings vs baseline, cache effectiveness)

Weekend Assignment: Production Multi-Agent System

Deploy enterprise-grade multi-agent system with:

Infrastructure:

- Dockerized services (agent-coordinator, workers, gateway)
- Kubernetes deployment (min 3 replicas per service)

- Redis for caching (6 GB memory) and message queue
- PostgreSQL for agent state persistence (with connection pooling)
- Kafka for event streaming (3 brokers, replication factor: 3)

Monitoring:

- Prometheus + Grafana dashboards (10+ custom metrics)
- OpenTelemetry distributed tracing (trace sampling: 10%)
- ELK stack for log aggregation (7-day retention)
- Custom metrics: latency (p50, p95, p99), cost per request, success rate, cache hit rate
- Alerting rules: PagerDuty/Slack (latency >3s, error rate >5%, cost spike >2x)

Performance:

- Load test: handle 200 concurrent requests
- 99th percentile latency <3 seconds (measure with k6 or Locust)
- Horizontal autoscaling based on CPU (target: 70%) and memory (target: 80%)
- Circuit breakers for failing services (failure threshold: 50%, timeout: 30s)
- Graceful degradation during peak load (queue requests, return cached responses)

Cost Tracking:

- Token usage per request (log to PostgreSQL)
- Cost attribution per user/agent (daily reports)
- Semantic caching with >60% hit rate (measure daily)
- Model routing for cost optimization (70% of requests to small model)

Deliverables:

- Architecture diagram with data flows (C4 model: Context, Container)
- Performance benchmarks: throughput (requests/sec), latency (p50, p95, p99), cost (\$/1K requests)
- Monitoring dashboards (Grafana screenshots with annotations)
- Load test results with analysis (k6 report, bottleneck identification)
- Incident response playbook (runbook for common failures)
- Cost analysis report (monthly cost projection, optimization recommendations)

WEEK 12 — ADVANCED CAPSTONE WEEK

Day 1: Problem Scoping & Architecture Design

Activities:

- User research and requirements gathering (interview 5+ stakeholders)
- Failure Mode and Effects Analysis (FMEA) - identify 10+ failure modes
- System design with trade-offs (CAP theorem considerations: Consistency, Availability, Partition tolerance)
- KPIs and success metrics definition (SMART criteria)

- Ethical Considerations and Bias Audit (fairness, transparency, accountability)
- Scalability Planning (10x, 100x growth scenarios with cost projections)
- Cost-Benefit Analysis (ROI calculation, break-even analysis)

Deliverables:

- Technical specification document (10+ pages): requirements, architecture, data models
- Architecture diagrams (C4 model: Context, Container, Component, Code levels)
- Data flow diagrams (DFD levels 0, 1, 2)
- API specifications (OpenAPI 3.0 with examples)
- Risk assessment matrix (likelihood × impact for 15+ risks)
- Evaluation criteria and acceptance tests (20+ test cases)

Day 2: Core System Implementation

Focus Areas:

- Multi-agent orchestration with error handling (try-catch, retry, fallback)
- Advanced RAG with validation pipeline (hybrid retrieval + hallucination detection)
- Custom tool development and integration (5+ tools with OpenAPI specs)
- Circuit Breakers and Graceful Degradation (failure handling)
- State Management and Persistence (database, caching)
- Security Implementation (AuthN with JWT, AuthZ with RBAC)

Technical Requirements:

- Implement at least 5 specialized agents (with distinct roles and skills)
- Build 3+ custom tools with validation (input validation, error handling)
- Integrate RAG with hybrid retrieval (dense + sparse fusion)
- Add caching layer with semantic similarity (FAISS, Redis)
- Implement retry logic with exponential backoff (max 3 retries, initial delay: 1s)
- Add authentication and authorization (JWT tokens, role-based access)

Day 3: Evaluation & Testing Framework

Testing Strategy:

- Automated Test Suite with pytest (100+ tests: unit, integration, E2E)
- Adversarial Testing (Jailbreak Attempts, Prompt Injection, 20+ attack vectors)
- Benchmark on Standard Datasets (MMLU for knowledge, HumanEval for code)
- Unit Tests (50+ tests for individual functions)
- Integration Tests (30+ tests for multi-component workflows)
- End-to-End Tests (20+ tests for complete user scenarios)
- Regression Testing for Agent Updates (prevent performance degradation)

- Property-Based Testing with Hypothesis (generate test cases automatically)
- Chaos Engineering for Resilience Testing (kill pods, inject latency)

Quality Metrics:

- Test coverage >85% (measure with pytest-cov)
- Automated testing in CI/CD (GitHub Actions, run on every commit)
- Performance benchmarks (latency, throughput, establish baselines)
- Security scanning results (Snyk, Trivy for vulnerabilities)

Exercise: Build comprehensive test suite:

- 50+ unit tests for agent logic (test individual functions in isolation)
- 30+ integration tests for multi-agent workflows (test agent interactions)
- 20+ end-to-end tests for user scenarios (test complete workflows)
- Adversarial tests: prompt injection (15 tests), jailbreak attempts (10 tests)
- Performance tests: latency (p95 <2s), throughput (>50 req/s)
- Load tests: 100 concurrent users (measure degradation)
- Regression tests for continuous integration (detect breaking changes)

Day 4: Deployment & Production Hardening

Deployment Pipeline:

- CI/CD Pipeline Setup (GitHub Actions or GitLab CI with multi-stage pipeline)
- Blue-Green Deployment Strategy (zero-downtime deployments)
- Canary Releases for Agent Updates (gradual rollout: 10% → 50% → 100%)
- Monitoring and Alerting (Prometheus + Grafana + PagerDuty)
- Rate Limiting and Abuse Prevention (100 requests/min per user)
- DDoS Protection and WAF Configuration (Cloudflare, AWS WAF)
- Disaster Recovery and Backup Strategy (RTO: 1 hour, RPO: 15 min)

Production Checklist:

- Container security scanning (Trivy, Snyk - no HIGH/CRITICAL vulns)
- Secret management (Vault, AWS Secrets Manager - rotate every 90 days)
- SSL/TLS configuration (TLS 1.3, A+ rating on SSL Labs)
- Log aggregation and retention (ELK stack, 30-day retention)
- Incident response plan (escalation matrix, communication plan)
- Backup and restore testing (quarterly drills)

Day 5: Documentation & Presentation

Documentation Artifacts:

System Documentation:

- Architecture overview (high-level design, technology stack)
- Component descriptions (responsibilities, interfaces, dependencies)

- Data models and schemas (ER diagrams, JSON schemas)
- API documentation (Swagger/OpenAPI with examples and error codes)

Operational Documentation:

- Deployment guide (step-by-step with prerequisites)
- Configuration management (environment variables, feature flags)
- Monitoring and alerting setup (dashboard setup, alert rules)
- Runbook for common issues (troubleshooting guide, 15+ scenarios)
- Disaster recovery procedures (backup, restore, failover)

User Documentation:

- User guide with examples (getting started, tutorials)
- API usage tutorials (authentication, endpoints, rate limits)
- Best practices (performance tips, security guidelines)
- FAQ (20+ common questions with answers)

Demo & Presentation:

- 15-minute video demo (screen recording with narration)
- Live system walkthrough (deployed on cloud, accessible URL)
- Failure case demonstrations (show error handling, recovery)
- Ethical considerations discussion (bias, privacy, fairness)

WEEKEND FINAL SUBMISSION: ADVANCED CAPSTONE PROJECT

Project Options

Option 1: Autonomous Research Assistant

Scope:

- Multi-agent research team: Coordinator, Searcher, Analyst, Validator, Writer (5 agents)
- Web + Academic Paper Search Integration: arXiv, Semantic Scholar, Google Scholar, PubMed
- Validated Knowledge Synthesis with Source Attribution (NLI-based fact-checking)
- Citation Verification and Fact-Checking (cross-reference 3+ sources per claim)
- Argument Graph Visualization (NetworkX, visualize claim relationships)
- Export to Multiple Formats: PDF, Markdown, LaTeX, HTML

Technical Requirements:

- Implement hybrid RAG: dense (sentence-transformers) + sparse (BM25) + graph (Neo4j)
- Use multi-hop retrieval for complex queries (max 4 hops)
- Add self-consistency validation (3 samples, majority vote)
- Include hallucination detection (NLI entailment score >0.8)

- Generate confidence scores per claim (0-100 scale)
- Track source provenance (maintain citation chain)

Option 2: Code Review & Refactoring System

Scope:

- Static Analysis Integration: pylint (code quality), mypy (type checking), bandit (security)
- Security Vulnerability Detection: Snyk, Trivy (scan dependencies)
- Automated Test Generation: property-based (Hypothesis), unit tests (pytest), integration tests
- Style Consistency Enforcement: black (formatting), flake8 (linting)
- Documentation Generation: docstrings (Google style), README (auto-generated)
- Refactoring Suggestions with Impact Analysis (code complexity, maintainability index)

Technical Requirements:

- Integrate with Git workflow (GitHub Actions, pre-commit hooks)
- Support multiple languages: Python, JavaScript, Java (extensible architecture)
- Implement automated PR comments (GitHub API integration)
- Generate test coverage reports (target: >80% coverage)
- Provide security severity ratings (CRITICAL, HIGH, MEDIUM, LOW)
- Track technical debt metrics (code smells, duplication, complexity)

Option 3: Intelligent Document Processing Pipeline

Scope:

- Multi-Modal Input: PDF, Images, Scans, Office Docs (DOCX, XLSX, PPTX)
- OCR with Layout Preservation: Tesseract, EasyOCR (handle tables, figures)
- Entity Extraction with Knowledge Graph (NER + relationship extraction)
- Compliance Checking: GDPR (data privacy), HIPAA (healthcare), SOC2 (security)
- Automated Report Generation with Templates (customizable Jinja2 templates)
- Document Classification and Routing (10+ document types)

Technical Requirements:

- Handle 10+ document types (PDF, DOCX, XLSX, PNG, JPG, etc.)
- Extract tables, figures, headers (preserve structure)
- Build knowledge graph of entities (Neo4j with 100+ entities)
- Implement compliance rule engine (20+ rules for each standard)
- Support batch processing (process 100 documents in <5 min)
- Provide audit trails (track all operations with timestamps)

Option 4: Adaptive Customer Support System

Scope:

- Intent Classification with Confidence Scoring (50+ intent types, accuracy >90%)
- Multi-Turn Conversation Management with Memory (handle 10+ turn conversations)
- Knowledge Base RAG with Hybrid Search (dense + sparse retrieval)
- Escalation Logic with Human Handoff (escalate when confidence <0.6)
- Sentiment Analysis and Empathy Response (detect frustration, adjust tone)
- Multi-Language Support (English, Spanish, French, German, Chinese)

Technical Requirements:

- Handle 50+ intent types (product info, troubleshooting, billing, etc.)
- Maintain conversation context (10+ turns with sliding window)
- Integrate with ticketing system (Zendesk, Jira Service Management)
- Implement sentiment-based routing (negative sentiment → priority queue)
- Provide response suggestions for human agents (auto-complete, templates)
- Track resolution metrics (CSAT, FCR, AHT)

MANDATORY DELIVERABLES FOR ALL PROJECTS

1. Technical Documentation

- Architecture diagram (C4 model: Context, Container, Component, Code)
- Component descriptions with responsibilities (each component documented)
- Data flow diagrams (DFD levels 0, 1, 2)
- Database schema if applicable (ER diagram, table definitions)
- API documentation (Swagger/OpenAPI 3.0 with examples)
- Deployment architecture (infrastructure diagram with annotations)

2. Code Repository

- Clean, well-documented code (docstrings, inline comments)
- Comprehensive test suite (>85% coverage, 100+ tests)
- CI/CD configuration (GitHub Actions or GitLab CI with linting, testing, deployment)
- Docker/Kubernetes manifests (Dockerfile, deployment.yaml, service.yaml)
- Infrastructure as Code (Terraform or CloudFormation for cloud resources)
- README with setup instructions (prerequisites, installation, usage)

3. Evaluation Report

- Validation methodology and metrics (describe evaluation approach)

- Performance benchmarks: latency (p50, p95, p99), throughput (req/s), cost (\$/1K requests)
- Hallucination analysis with examples (identify failure cases)
- Cost analysis: per request, per user, monthly projection
- A/B test results if applicable (statistical significance, p-value <0.05)
- Error analysis with failure categories (false positives, false negatives, edge cases)

4. Demo & Presentation

- 15-minute video demo with narration (Loom, YouTube)
- Live system deployment (accessible URL, credentials provided)
- Failure case demonstrations (show error handling, recovery mechanisms)
- Ethical considerations and limitations (bias, privacy, fairness)
- Comparison with existing solutions (feature comparison, performance benchmarks)
- Future roadmap (planned improvements, scaling strategy)

5. Novel Contributions

At least TWO unique techniques not in mainstream courses:

Examples:

- Novel prompting strategy (e.g., dynamic few-shot selection)
- Custom evaluation metric (e.g., task-specific quality score)
- Innovative agent coordination pattern (e.g., auction-based task allocation)
- New retrieval method (e.g., hybrid dense-sparse-graph fusion)
- Unique fine-tuning approach (e.g., curriculum learning with difficulty scoring)
- Comparative analysis with existing solutions (ablation study showing improvements)
- Ablation study showing contribution impact (measure performance gain from each component)

UNIQUE DIFFERENTIATORS IN THIS CURRICULUM

Topics NOT Found in Standard Courses

Agent Systems:

- Agent mesh networks with dynamic role assignment
- Byzantine fault tolerance in multi-agent systems
- Auction-based task allocation (Vickrey, first-price sealed-bid)
- Emergent behavior detection and control
- Swarm intelligence principles (particle swarm optimization, ant colony)

Retrieval & Knowledge:

- Learned sparse retrieval (SPLADE, uniCOIL)

- Temporal knowledge graphs with time-aware reasoning
- Graph-of-Thoughts reasoning (GoT)
- Community detection for topic clustering (Louvain, label propagation)
- Causal validation for reasoning chains

Validation & Quality:

- Monte Carlo Dropout for uncertainty in LLMs
- Semantic similarity validation with BERTScore, MoverScore
- Hallucination detection using NLI models (RoBERTa, DeBERTa)
- Agent trajectory validation (logical step verification)
- Self-consistency sampling with majority vote

Fine-Tuning & Optimization:

- Mixture of LoRAs (MoLoRA) for multi-task learning
- Direct Preference Optimization (DPO) as alternative to RLHF
- Algorithm distillation for in-context learning
- Prompt compression with LLMingua
- Model merging techniques (SLERP, TIES-Merging)

Infrastructure & Scaling:

- Speculative decoding for inference speedup (2-3x faster)
- PagedAttention for memory efficiency (reduce memory by 50%)
- Event sourcing for agent state (CQRS pattern)
- OpenTelemetry for agent tracing (distributed tracing)
- Model routing based on query complexity (cost optimization)

Advanced Reasoning:

- Tree-of-Thoughts exploration (ToT with beam search)
- Graph-of-Thoughts reasoning (GoT with graph traversal)
- Retrieval-Augmented Thought (RAT) patterns
- Self-Ask prompting (iterative question decomposition)
- Chain-of-thought distillation (compress reasoning into smaller models)

Security & Safety:

- WebAssembly for sandboxed execution (browser-based isolation)
- AST-based security validation (detect malicious code patterns)
- Byzantine fault tolerance (tolerate malicious agents)
- Circuit breakers for agent systems (prevent cascade failures)
- Adversarial robustness testing (jailbreak, prompt injection)

ASSESSMENT CRITERIA

Technical Excellence (40%)

- Code quality and architecture (clean code, design patterns)

- Innovation in approach (novel techniques, creative solutions)
- System robustness and error handling (graceful degradation, retry logic)
- Performance optimization (latency, throughput, cost)
- Security implementation (authentication, authorization, encryption)

Validation & Testing (30%)

- Comprehensive evaluation methodology (multiple metrics, datasets)
- Proper use of validation techniques (cross-validation, hallucination detection)
- Edge case handling (boundary conditions, error scenarios)
- Regression testing (prevent breaking changes)
- Adversarial robustness (resist attacks, maintain reliability)

Production Readiness (20%)

- Deployment quality (CI/CD, containerization, orchestration)
- Monitoring and observability (metrics, logs, traces)
- Documentation completeness (technical, operational, user docs)
- Scalability considerations (horizontal scaling, load balancing)
- Cost optimization (caching, routing, efficient inference)

Innovation (10%)

- Novel techniques implemented (2+ unique contributions)
- Creative problem-solving (unconventional approaches)
- Contributions beyond curriculum (research potential)
- Research potential (publishable insights, open-source contributions)

GRADING RUBRIC

A+ (95-100): Exceptional work with 3+ novel contributions, production-ready system, comprehensive evaluation, exceeds all requirements

A (90-94): Excellent work with 2+ novel contributions, near-production quality, thorough evaluation, meets all requirements

A- (85-89): Strong work with 1-2 novel contributions, good quality, solid evaluation, meets most requirements

B+ (80-84): Good work with 1 novel contribution, functional system, adequate evaluation, meets core requirements

B (75-79): Satisfactory work, working system, basic evaluation, meets minimum requirements

Below 75: Incomplete or insufficient work, missing key components

End of Curriculum

This comprehensive 6-week advanced certification covers cutting-edge topics in agentic AI, preparing learners to build production-grade systems with novel contributions to the field.

Technical Excellence (40%)

- Code quality and architecture
- Innovation in approach
- System robustness and error handling
- Performance optimization
- Security implementation

Validation & Testing (30%)

- Comprehensive evaluation methodology
- Proper use of validation techniques
- Edge case handling
- Regression testing
- Adversarial robustness

Production Readiness (20%)

- Deployment quality
- Monitoring and observability
- Documentation completeness
- Scalability considerations
- Cost optimization

Innovation (10%)

- Novel techniques implemented
- Creative problem-solving
- Contributions beyond curriculum
- Research potential